Dissecting Transmission: The Beta-Poisson Model

Francesca Hammerstein, Nathan Hogg, Manya Kaushik

2026-04-24

1. Data Preparation

Source: Adam et al. "Clustering and superspreading potential of SARS-CoV-2 infections in Hong Kong." *Nature Medicine* (2020). doi: https://doi.org/10.1038/s41591-020-1092-0

```r
pairs <- read.csv("transmission_pairs.csv")

# Count secondary cases per infector
infector_counts <- table(pairs$infector.case)

# Identify terminal cases (infectees who never infect anyone) - these are zeros
all_infectees <- unique(pairs$infectee.case)
all_infectors <- unique(pairs$infector.case)
terminal_cases <- setdiff(all_infectees, all_infectors)

# Build full secondary case vector including zeros
secondary_cases <- c(as.integer(infector_counts), rep(0, length(terminal_cases)))

# Frequency table
freq_table <- as.data.frame(table(secondary_cases))
colnames(freq_table) <- c("secondary_cases", "frequency")
freq_table$secondary_cases <- as.integer(as.character(freq_table$secondary_cases))
print(freq_table)
```

```
##   secondary_cases frequency
## 1               0       153
## 2               1        57
## 3               2        18
## 4               3         6
## 5               4         5
## 6               5         1
## 7               6         2
## 8              10         1
## 9              11         1
```

```r
# Expanded raw data vector for model fitting
raw_data <- rep(freq_table$secondary_cases, freq_table$frequency)

cat("\nTotal individuals:", length(raw_data), "\n")
```

```
##
## Total individuals: 244
```

```r
cat("Mean secondary cases (R):", mean(raw_data), "\n")
```

```
## Mean secondary cases (R): 0.692623
```

```r
cat("Variance:", var(raw_data), "\n")
```

```
## Variance: 1.909246
```

```r
cat("Proportion of zeros:", mean(raw_data == 0), "\n")
```

```
## Proportion of zeros: 0.6270492
```

2. Model Fitting

2.1 Poisson MLE (Francesca)

```r
lambda_poi <- mean(raw_data)
loglik_poi <- sum(dpois(raw_data, lambda_poi, log = TRUE))
aic_poi <- 2 * 1 - 2 * loglik_poi

cat(" Poisson MLE Results \n")
```

```
##  Poisson MLE Results
```

```r
cat("Lambda:", lambda_poi, "\n")
```

```
## Lambda: 0.692623
```

```r
cat("Log-likelihood:", loglik_poi, "\n")
```

```
## Log-likelihood: -320.7387
```

```r
cat("AIC:", aic_poi, "\n")
```

```
## AIC: 643.4775
```

2.2 Geometric MLE (Francesca)

```r
lambda_geo <- mean(raw_data)
loglik_geo <- sum(dgeom(raw_data, prob = 1 / (1 + lambda_geo), log = TRUE))
aic_geo <- 2 * 1 - 2 * loglik_geo

cat(" Geometric MLE Results \n")
```

```
##  Geometric MLE Results
```

```r
cat("Lambda:", lambda_geo, "\n")
```

```
## Lambda: 0.692623
```

```r
cat("Log-likelihood:", loglik_geo, "\n")
```

```
## Log-likelihood: -279.4219
```

```r
cat("AIC:", aic_geo, "\n")
```

```
## AIC: 560.8439
```

2.3 Negative Binomial MLE (Manya)

```r
lambda_nb <- mean(raw_data)

nb_negloglik <- function(theta, data) {
  -sum(dnbinom(data, size = theta, mu = mean(data), log = TRUE))
}

nb_fit <- optim(par = 1, fn = nb_negloglik, data = raw_data,
                method = "L-BFGS-B", lower = 1e-6)

theta_hat <- nb_fit$par
loglik_nb <- -nb_fit$value
aic_nb <- 2 * 2 - 2 * loglik_nb

cat(" Negative Binomial MLE Results \n")
```

```
##  Negative Binomial MLE Results
```

```r
cat("Lambda (mean):", lambda_nb, "\n")
```

```
## Lambda (mean): 0.692623
```

```r
cat("Theta (overdispersion):", theta_hat, "\n")
```

```
## Theta (overdispersion): 0.5576684
```

```r
cat("Log-likelihood:", loglik_nb, "\n")
```

```
## Log-likelihood: -276.2905
```

```r
cat("AIC:", aic_nb, "\n")
```

```
## AIC: 556.581
```

2.4 Zero-Inflated Poisson MLE (Nathan)

```r
# Log-Likelihood Function
zip_loglik <- function(params, data) {
  lambda <- params[1]
  sigma <- params[2]

  if (lambda <= 0 || sigma <= 0 || sigma >= 1) return(-Inf)

  zeros <- data[data == 0]
  nonzeros <- data[data > 0]

  ll_zeros <- length(zeros) * log(sigma + (1 - sigma) * exp(-lambda))
  ll_nonzeros <- sum(log(1 - sigma) + dpois(nonzeros, lambda, log = TRUE))

  return(ll_zeros + ll_nonzeros)
}

zip_negloglik <- function(params, data) {
  return(-zip_loglik(params, data))
}
```

```r
fit_zip <- function(data) {
  p_zero <- mean(data == 0)
  start_sigma <- max(0.01, min(0.99, p_zero - 0.1))
  start_lambda <- mean(data) / (1 - start_sigma)

  result <- optim(
    par = c(start_lambda, start_sigma),
    fn = zip_negloglik,
    data = data,
    method = "L-BFGS-B",
    lower = c(1e-6, 1e-6),
    upper = c(Inf, 1 - 1e-6),
    hessian = TRUE
  )

  lambda_hat <- result$par[1]
  sigma_hat <- result$par[2]
  loglik <- -result$value
  aic <- 2 * 2 - 2 * loglik

  cat(" ZIP MLE Results \n")
  cat("Lambda (Poisson Mean):", lambda_hat, "\n")
  cat("Sigma (Zero-Inflation):", sigma_hat, "\n")
  cat("Log-Likelihood:", loglik, "\n")
  cat("AIC:", aic, "\n")
  cat("Overall mean (lambda * (1 - sigma)):", lambda_hat * (1 - sigma_hat), "\n")

  return(list(lambda = lambda_hat, sigma = sigma_hat, loglik = loglik, aic = aic))
}

zip_result <- fit_zip(raw_data)
```

```
##  ZIP MLE Results
## Lambda (Poisson Mean): 1.398481
## Sigma (Zero-Inflation): 0.5047316
## Log-Likelihood: -295.6031
## AIC: 595.2061
## Overall mean (lambda * (1 - sigma)): 0.6926233
```

```r
# Predicted vs observed zeros
n <- length(raw_data)
predicted_zero_prop <- zip_result$sigma + (1 - zip_result$sigma) * exp(-zip_result$lambda)
cat("\nObserved zero proportion:", mean(raw_data == 0), "\n")
```

```
##
## Observed zero proportion: 0.6270492
```

```r
cat("Predicted zero proportion:", predicted_zero_prop, "\n")
```

```
## Predicted zero proportion: 0.627049
```

2.5 Beta-Poisson MLE (Nathan)

```r
bp_pmf <- function(x, lambda, phi, nu) {
  N <- 1 / nu
```

```r
  alpha1 <- lambda * phi
  alpha2 <- (N - lambda) * phi

  if (alpha1 <= 0 || alpha2 <= 0 || N <= lambda) return(rep(-Inf, length(x)))

  sapply(x, function(k) {
    f <- function(p) dpois(k, lambda = p * N) * dbeta(p, alpha1, alpha2)
    val <- tryCatch(integrate(f, 0, 1, rel.tol = 1e-8)$value,
                    error = function(e) 0)
    if (val <= 0) return(-Inf)
    log(val)
  })
}

bp_negloglik <- function(params, data, lambda) {
  phi <- params[1]
  nu <- params[2]
  if (phi <= 0 || nu <= 0 || nu >= 1/lambda) return(1e10)
  ll <- sum(bp_pmf(data, lambda, phi, nu))
  if (is.na(ll) || is.infinite(ll)) return(1e10)
  return(-ll)
}

fit_bp <- function(data) {
  lambda <- mean(data)
  best <- list(value = 1e10)

  for (sp in c(0.1, 0.5, 1, 5)) {
    for (sn in c(0.01, 0.1, 0.5)) {
      sv <- sn / lambda
      if (sv >= 1/lambda) next

      res <- tryCatch(
        optim(c(sp, sv), bp_negloglik, data = data, lambda = lambda,
              method = "L-BFGS-B",
              lower = c(1e-6, 1e-8),
              upper = c(100, 1/lambda - 1e-6)),
        error = function(e) list(value = 1e10))

      if (res$value < best$value) best <- res
    }
  }

  phi_hat <- best$par[1]
  nu_hat <- best$par[2]
  N_hat <- 1 / nu_hat
  loglik <- -best$value
  aic <- 6 - 2 * loglik

  cat(" Beta-Poisson MLE Results \n")
  cat("Lambda (mean):", lambda, "\n")
  cat("Phi (overdispersion):", phi_hat, "\n")
  cat("Nu (inverse contacts):", nu_hat, "\n")
```

```r
  cat("N (mean contacts):", N_hat, "\n")
  cat("Log-likelihood:", loglik, "\n")
  cat("AIC:", aic, "\n")

  if (nu_hat < 0.01) {
    cat("\nNote: Nu ~ 0 (N -> Inf), beta-Poisson has converged to its\n")
    cat("negative binomial limit with theta =", 1/phi_hat, "\n")
  }

  return(list(lambda = lambda, phi = phi_hat, nu = nu_hat,
              N = N_hat, loglik = loglik, aic = aic))
}

bp_result <- fit_bp(raw_data)
```

```
##  Beta-Poisson MLE Results
## Lambda (mean): 0.692623
## Phi (overdispersion): 0.804689
## Nu (inverse contacts): 0.0004845876
## N (mean contacts): 2063.61
## Log-likelihood: -276.2956
## AIC: 558.5912
##
## Note: Nu ~ 0 (N -> Inf), beta-Poisson has converged to its
## negative binomial limit with theta = 1.242716
```

3. Model Comparison (Francesca)

```r
comparison <- data.frame(
  Model = c("Poisson", "Geometric", "Neg Binomial", "ZIP", "Beta-Poisson"),
  Parameters = c(1, 1, 2, 2, 3),
  LogLik = c(loglik_poi, loglik_geo, loglik_nb,
             zip_result$loglik, bp_result$loglik),
  AIC = c(aic_poi, aic_geo, aic_nb,
          zip_result$aic, bp_result$aic)
)
comparison <- comparison[order(comparison$AIC), ]

cat(" AIC Model Comparison \n")
```

```
##  AIC Model Comparison
```

```r
print(comparison, row.names = FALSE)
```

```
##         Model Parameters    LogLik      AIC
##  Neg Binomial          2 -276.2905 556.5810
##  Beta-Poisson          3 -276.2956 558.5912
##     Geometric          1 -279.4219 560.8439
##           ZIP          2 -295.6031 595.2061
##       Poisson          1 -320.7387 643.4775
```

4. Branching Process Simulation (Nathan)

```r
simulate_outbreak <- function(rfunc, n_gen = 20, n_index = 1) {
  cases_per_gen <- numeric(n_gen)
  cases_per_gen[1] <- n_index
  total_cases <- n_index

  for (g in 2:n_gen) {
    if (cases_per_gen[g - 1] == 0) break
    new_cases <- sum(rfunc(cases_per_gen[g - 1]))
    cases_per_gen[g] <- new_cases
    total_cases <- total_cases + new_cases
  }
  return(list(total = total_cases, by_gen = cases_per_gen))
}

# Random draw functions
r_zip <- function(n) {
  is_zero <- rbinom(n, 1, zip_result$sigma)
  ifelse(is_zero, 0, rpois(n, zip_result$lambda))
}

r_betapois <- function(n) {
  N <- 1 / bp_result$nu
  alpha1 <- bp_result$lambda * bp_result$phi
  alpha2 <- (N - bp_result$lambda) * bp_result$phi
  p <- rbeta(n, alpha1, alpha2)
  rpois(n, p * N)
}

# Simulate
n_sims <- 10000
n_gen <- 20
zip_sizes <- numeric(n_sims)
bp_sizes <- numeric(n_sims)

set.seed(42)
for (i in 1:n_sims) {
  zip_sizes[i] <- simulate_outbreak(r_zip, n_gen)$total
  bp_sizes[i] <- simulate_outbreak(r_betapois, n_gen)$total
}

cat(" Branching Process Results (", n_sims, "simulations) \n\n")
```

```
##  Branching Process Results ( 10000 simulations)
```

```r
cat("--- ZIP ---\n")
```

```
## --- ZIP ---
```

```r
cat("Mean outbreak size:", mean(zip_sizes), "\n")
```

```
## Mean outbreak size: 3.2154
```

```r
cat("Median outbreak size:", median(zip_sizes), "\n")
```

```
## Median outbreak size: 1
```

```r
cat("Max outbreak size:", max(zip_sizes), "\n")
```

```
## Max outbreak size: 129
```

```r
cat("P(extinction by gen", n_gen, "):", mean(zip_sizes < 50), "\n")
```

```
## P(extinction by gen 20 ): 0.9967
```

```r
cat("P(outbreak > 10 cases):", mean(zip_sizes > 10), "\n\n")
```

```
## P(outbreak > 10 cases): 0.0643
```

```r
cat("--- Beta-Poisson ---\n")
```

```
## --- Beta-Poisson ---
```

```r
cat("Mean outbreak size:", mean(bp_sizes), "\n")
```

```
## Mean outbreak size: 3.2581
```

```r
cat("Median outbreak size:", median(bp_sizes), "\n")
```

```
## Median outbreak size: 1
```

```r
cat("Max outbreak size:", max(bp_sizes), "\n")
```

```
## Max outbreak size: 226
```

```r
cat("P(extinction by gen", n_gen, "):", mean(bp_sizes < 50), "\n")
```

```
## P(extinction by gen 20 ): 0.9955
```

```r
cat("P(outbreak > 10 cases):", mean(bp_sizes > 10), "\n\n")
```

```
## P(outbreak > 10 cases): 0.0609
```

```r
# Superspreading analysis
R <- mean(raw_data)
ss_threshold <- qpois(0.99, R)
cat(" Superspreading Analysis \n")
```

```
##  Superspreading Analysis
```

```r
cat("R (mean):", R, "\n")
```

```
## R (mean): 0.692623
```

```r
cat("Superspreading threshold (99th %ile of Poi(R)):", ss_threshold, "\n")
```

```
## Superspreading threshold (99th %ile of Poi(R)): 3
```

```r
cat("Observed superspreaders:", sum(raw_data > ss_threshold), "/", length(raw_data), "\n")
```

```
## Observed superspreaders: 10 / 244
```

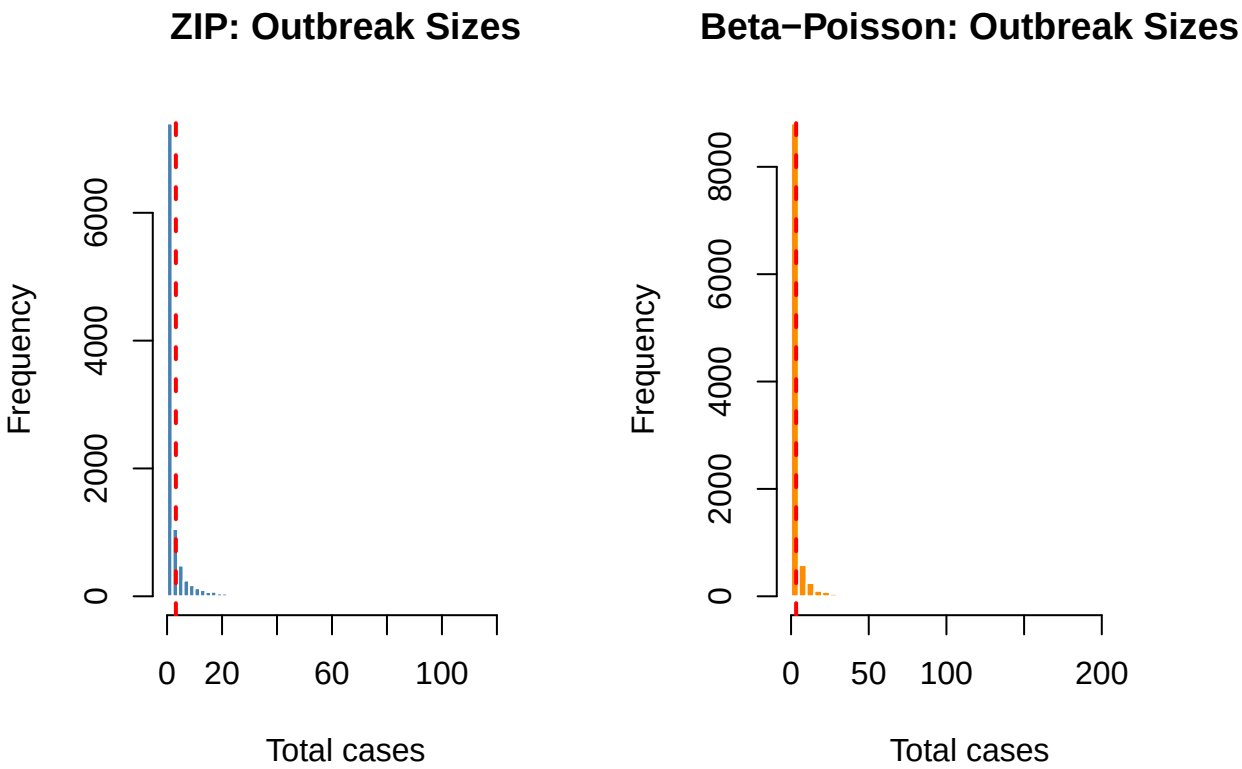
```r
cat("Observed superspreader proportion:", mean(raw_data > ss_threshold), "\n")
```

```
## Observed superspreader proportion: 0.04098361
```

```r
# Plots
par(mfrow = c(1, 2))
hist(zip_sizes, breaks = 50, main = "ZIP: Outbreak Sizes",
     xlab = "Total cases", col = "steelblue", border = "white")
```

```r
abline(v = mean(zip_sizes), col = "red", lwd = 2, lty = 2)

hist(bp_sizes, breaks = 50, main = "Beta-Poisson: Outbreak Sizes",
     xlab = "Total cases", col = "darkorange", border = "white")
abline(v = mean(bp_sizes), col = "red", lwd = 2, lty = 2)
```



5. Original Analysis: Stratified Transmission by Setting (Manya)

```r
# Assign each infector their primary transmission setting
infector_settings <- aggregate(cluster.risk ~ infector.case, data = pairs,
                               FUN = function(x) names(sort(table(x), decreasing = TRUE))[1])

infector_sec <- data.frame(
  infector = names(infector_counts),
  secondary_cases = as.integer(infector_counts)
)

infector_info <- merge(infector_sec, infector_settings,
                       by.x = "infector", by.y = "infector.case", all.x = TRUE)

cat(" Counts per Setting \n")
```

```
##  Counts per Setting
```

```r
print(table(infector_info$cluster.risk))
```

```
##
## family social travel   work
##     61     17      1     12
```

```r
cat("\n Summary by Setting \n")
```

```
##
##  Summary by Setting
```

```r
for (setting in c("family", "social", "work")) {
  subset_data <- infector_info$secondary_cases[infector_info$cluster.risk == setting]
  if (length(subset_data) < 5) next

  lambda_s <- mean(subset_data)

  nb_fit_s <- tryCatch(
    optim(par = 1, fn = function(theta, data) {
      -sum(dnbinom(data, size = theta, mu = mean(data), log = TRUE))
    }, data = subset_data, method = "L-BFGS-B", lower = 1e-6),
    error = function(e) NULL
  )

  if (!is.null(nb_fit_s)) {
    cat("\n---", setting, "(n =", length(subset_data), ") ---\n")
    cat("Mean (R):", lambda_s, "\n")
    cat("Theta:", nb_fit_s$par, "\n")
    cat("Variance:", var(subset_data), "\n")
    cat("Overdispersion ratio (var/mean):", var(subset_data) / lambda_s, "\n")
  }
}
```

```
##
## --- family (n = 61 ) ---
## Mean (R): 1.47541
## Theta: 50828.34
## Variance: 0.5868852
## Overdispersion ratio (var/mean): 0.3977778
##
## --- social (n = 17 ) ---
## Mean (R): 3.352941
## Theta: 2.167291
## Variance: 10.24265
## Overdispersion ratio (var/mean): 3.054825
##
## --- work (n = 12 ) ---
## Mean (R): 1.75
## Theta: 70376.91
## Variance: 1.840909
## Overdispersion ratio (var/mean): 1.051948
```

```r
# Fisher's exact test: association between transmission setting and age group
tbl <- table(pairs$cluster.risk, pairs$agegroup.infector)

cat(" Fisher's Exact Test \n")
```

```
##  Fisher's Exact Test
```

```r
cat("H0: No association between cluster risk type and infector age group\n")
```

```
## H0: No association between cluster risk type and infector age group
```

```r
cat("H1: Association exists\n\n")
```

```
## H1: Association exists
```

```r
fisher_result <- fisher.test(tbl, simulate.p.value = TRUE, B = 10000)
print(fisher_result)
```

```
##
##  Fisher's Exact Test for Count Data with simulated p-value (based on
##  10000 replicates)
##
## data:  tbl
## p-value = 0.0008999
## alternative hypothesis: two.sided
```

```r
cat("\n Proportions of Setting by Age Group \n")
```

```
##
##  Proportions of Setting by Age Group
```

```r
print(round(prop.table(tbl, margin = 2), 3))
```

```
##
##              1     3     5     6     7     8     9    10    11    12    13
##   family 1.000 1.000 0.600 0.917 0.261 0.467 0.667 0.364 0.800 0.562 0.562
##   social 0.000 0.000 0.400 0.083 0.565 0.500 0.222 0.364 0.100 0.000 0.250
##   travel 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.062 0.000
##   work   0.000 0.000 0.000 0.000 0.174 0.033 0.111 0.273 0.100 0.375 0.188
##
##             14    15    16    18
##   family 0.833 1.000 0.625 1.000
##   social 0.000 0.000 0.375 0.000
##   travel 0.000 0.000 0.000 0.000
##   work   0.167 0.000 0.000 0.000
```